The Boundary of Graph Sparsification for Community Detection

Mohammad Dindoost, Bavan DivaaniAazar, Ioannis Koutis, and David A. Bader

Abstract—

Index Terms—Graph sparsification, community detection, modularity, evaluation methodology, graph reduction.

I. Introduction

Community detection is a fundamental task in network analysis [1], [2]. Given a graph, the goal is to partition its vertices into groups that are densely connected internally and sparsely connected to the rest of the graph, and the resulting partitions are used to summarize structure, to identify functional units, and to organize downstream computation. The task is computationally demanding, and the most widely used methods, including modularity optimization [3], [4], [5] and flow-based approaches [6], have running times that scale with the number of edges rather than with the number of vertices. On the graphs to which these methods are now routinely applied, that quantity is large [7].

Reducing the vertex set is not available as a remedy, because a community assignment is required for every vertex; removing vertices alters the problem rather than reducing it. Reducing the edge set is available. Graph sparsification constructs a subgraph on the same vertex set with substantially fewer edges while preserving properties of the original [8], [9]. If the properties preserved include community structure, then detection can be performed on the sparsified graph at lower cost and without loss of quality. That premise has been pursued for fifteen years, beginning with Satuluri et al. [10], and the resulting methods are now implemented in standard graph libraries.

Satuluri et al. [10] introduced local similarity sparsification, in which each vertex retains the incident edges whose endpoints have the highest Jaccard similarity of neighbourhoods. Evaluating four clustering algorithms on seven networks, they reported speedups "often in the range 10-50, with little or no deterioration in the quality of the resulting clusters," and stated that "for at least two of the four clustering algorithms, our sparsification consistently enables higher clustering accuracies."

The conditions under which that result was obtained are specific. The two algorithms for which accuracy improved are fixed-k, balance-constrained cut partitioners. Accuracy is measured as agreement with external ground-truth communities rather than by any objective computed on the graph. The reported speedups are relative to clustering runs requiring hours on graphs of up to 1.2×10^8 edges. The networks on which the gains are largest have average degrees of 65, 76 and 94, and two of them are derived from high-throughput assays known to contain false-positive edges. The same paper reports a synthetic sweep in which the method "actually outperforms the original clustering starting from degree 50." To our knowledge, no subsequent work examines that threshold.

Subsequent work extended the recommendation beyond these conditions. Sparsify-then-detect has since been applied to modularity optimizers as well as to cut partitioners, to sparse graphs as well as to dense ones, and evaluated against graph-internal objectives as well as against external references. The degree threshold reported in the original sweep does not appear in this later literature, and the conditions attached to the original result are not generally restated with it.

A measurement requirement is also frequently omitted, and the omission has a direct bearing on what can be concluded. A sparsifier intended for community detection removes edges lying between communities in preference to edges lying within them. Quality measures for a partition, including modularity and conductance, penalize exactly the between-community edges that such a sparsifier removes. A partition evaluated on the sparsified graph is therefore evaluated against a graph from which its penalty term has been partially deleted, and the resulting quantity reflects the sparsifier's selectivity rather than the extent to which community structure was preserved. The comparison that answers the intended question evaluates the partition obtained from the sparsified graph and the partition obtained from the original graph on the same graph, namely the original one.

This requirement is stated in the 2011 paper, whose authors write that they "cannot simply measure the conductance using the very same sparsified graph, since that would tell us nothing about how well the sparsified graph retained the cluster structure in the original graph," and who report every quality figure on the original graph accordingly. It is absent from much of the work that followed, and from the released evaluation code of a widely used sparsification benchmark.

The effect of the omission is large. Across 63 controlled configurations spanning four detection algorithms, two sparsifier families and seven networks, evaluating on the sparsified graph rather than on the original reverses the sign of the conclusion in 45 of them. In the most extreme

The authors are with the Department of Computer Science, New Jersey Institute of Technology, Newark, NJ, USA. E-mail: {md724, bd344, ikoutis, bader}@njit.edu

configuration, a pipeline that reduces modularity by 0.40 under the correct evaluation appears to increase it by 0.39 under the incorrect one. Under such an evaluation a boundary cannot be located, since improvement is reported throughout the parameter range.

We therefore rebuilt the evaluation before re-asking the question. Partitions are always scored on the original graph. Comparisons against a stochastic baseline are given the same wall-clock budget as the pipeline they are compared with, and are also checked against a fixed-restart baseline, because a runtime-matched control is weak precisely when the pipeline is fast. Because sparsification fragments partitions, and because best-match and information-theoretic recovery scores rise mechanically with the number of clusters, every recovery comparison is matched for granularity against a partition of the original graph at the same cluster count, and is referenced to a chance floor computed from size-matched random partitions. Because a heavy-tailed degree sequence alone can produce several of the effects attributed to community structure, the central mechanistic claims are checked against degree-preserving randomizations of the same graphs.

With that instrument we ran 23 controlled experiments over both sparsifier families that have ever claimed quality gains, degree-based and similarity-based, the latter being the 2011 method itself; four detection algorithms drawn from two families; seventeen real networks of up to 25 million edges; and a synthetic sweep of the one variable the original authors identified as decisive. The design locates a boundary rather than confirming or refuting a claim, and it is reported as such. Predictions and stopping rules were fixed before each experiment ran, and several of our own hypotheses did not survive them.

Our results locate a boundary, and the transition across it is abrupt.

On one side of it sparsification does not improve detection. For modularity-family detectors with free granularity, on graphs with average degree between 5 and 33, no sparsifier we tested improves modularity measured on the original graph beyond the variation of a small number of random restarts, at any retention level that preserves quality, under any of the four algorithms examined. Nor is there a speed benefit. Measured end to end, including the cost of the sparsifier itself, the pipeline requires between $0.87\times$ and $1.35\times$ the time of clustering the original graph directly, and at aggressive retention it is slower, because a fragmented graph presents the optimizer with more work rather than less. Every apparent gain we found in this regime, in modularity, in ground-truth recovery, and in the one case where a sparsifier's edge-selection signal is demonstrably structure-aware rather than degree-driven, dissolves when granularity is matched, when chance is subtracted, or when the baseline is given an equal compute budget.

On the other side of the boundary sparsification does improve detection, under the conditions the original authors described. On synthetic graphs with planted communities, holding the number of communities fixed by construction so that no granularity effect is possible, similarity-based sparsification raises both the objective measured on the original graph and chance-corrected agreement with the planted labels once the average degree passes approximately 50. The effect is absent at average degree 11 and 25, present in two thirds of configurations at 49, and present in all of them at 102 and 221. It survives the worst case over ten independent runs of the partitioner, and it is not reproduced by degree-based or uniform sparsification at identical retention, so it is a property of the similarity signal rather than of the edge budget. Injecting spurious edges makes the gain grow monotonically, which is the signature the denoising explanation predicts.

What decides the outcome is not which sparsifier is used. It is the average degree of the graph, the family of the detection algorithm, since fixed-k balanced partitioners gain where free-granularity modularity optimizers do not, and whether quality is measured against an external reference or against an objective defined on the graph whose edges were deleted. The mechanism is consistent with this. The quantity governing a degree-based sparsifier's effect on modularity is the separation between the scores it assigns to within-community and between-community edges, and we show by direct intervention that this quantity is controlled by where high-degree-product edges sit relative to community boundaries, at fixed degree sequence, fixed partition and fixed modularity.

Two qualifications attach to this result. Sparsification repairs a weaker detector rather than surpassing a stronger one: a resolution-tuned modularity optimizer on the untouched graph outperforms every sparsified pipeline we measured. And with an exact similarity computation the quality gain is not a speed gain, because the sparsifier costs more than the detection it accelerates.

This paper contributes the following.

- The boundary. A degree threshold near average degree 50 and an algorithm-family split, established under controls that exclude granularity by construction, together with the negative territory delimited across two sparsifier families, four detection algorithms and seventeen networks.
- An evaluation protocol, and the measured cost of omitting it. Each control is motivated by a specific failure it catches, and the consequence of the central omission is quantified at 45 sign reversals in 63 configurations. We also introduce a permuted-weight control for similarity-weighted pipelines, which shows that the benefit attributed to similarity weighting is reproduced, and in one case exceeded, when the weights are randomly permuted across edges.
- A mechanism for the boundary. A decomposition of the score-separation quantity into a sorting term and a granularity term across seventeen networks, and a direct causal test in which that quantity is steered through more than a full sign change while degree sequence, partition, intra-edge fraction and

modularity are all held exactly fixed.

- An open phenomenon. In several controlled settings the partitions that best recover reference communities are not the partitions that score best on the objective, and the two criteria move in opposite directions within the same configuration. We report this as an observation rather than a result, and argue that it is the most consequential open question the boundary exposes.

II. Background and Setting

III. What Honest Evaluation Requires

IV. The Mechanism

V. Below the Boundary: Where Sparsification Does Not Help

VI. The Boundary

Everything to this point has been measured on real networks whose average degree lies between 5.5 and 32.6, and on that territory the answer is uniformly negative. That range is not a neutral choice. It is the range in which sparsification for community detection is normally studied, and it lies entirely below the one quantitative boundary the founding work names [10]: a synthetic sweep in which local similarity sparsification "actually outperforms the original clustering starting from degree 50." A negative result confined to one side of a boundary answers half the question. This section supplies the other half.

A. Design

Three properties are required of the setting. Average degree must vary over an order of magnitude while everything else is held fixed. Community labels must be known rather than inferred. And the granularity artifact that accounts for most of the apparent gains in this literature must be removable by construction rather than by control.

Synthetic benchmarks with planted communities supply the first two. A fixed-k partitioner supplies the third. If the number of communities is an input to the algorithm, then the sparsified and unsparsified arms return the same number of communities by definition, and no difference between them can be attributed to one partition being finer than the other.

We generate LFR benchmark graphs [11] with $n = 10^4$ nodes and degree exponent 2.1, matching the generator settings of the original sweep, at nominal average degrees of 10, 25, 50, 100 and 200, at two mixing levels, with three graphs per configuration. Realized degree and mixing are recorded per graph, and all results are reported against realized values. Three sparsifiers are applied at matched realized retention: local similarity sparsification, degree-based sparsification, and uniform random edge deletion. Detection is performed both by a free-granularity modularity optimizer and, through the pymetis bindings, by Metis, a fixed-k balance-constrained cut partitioner from the family the original accuracy claim concerns. Quality is scored on the original graph throughout. Recovery is measured against the planted labels with chance-corrected indices and a size-matched random-partition floor.

TABLE I

Fixed-k partitioner (Metis), realized mixing ≈ 0.6. Counts are configurations out of twelve in which similarity sparsification improves the quantity named, with the mean change beside them. Modularity is measured on the original graph. Because k is an input to the partitioner, the sparsified and unsparsified arms return identical community counts, so no granularity effect is possible.

Avg. degree	$\Delta Q > 0$	mean ΔQ	ΔAMI > 0	mean ΔAMI
11.2	0/12	−0.103	0/12	−0.216
24.6	0/12	−0.046	6/12	−0.072
49.4	8/12	−0.016	9/12	+0.042
101.9	10/12	+0.006	12/12	+0.116
220.9	12/12	+0.009	12/12	+0.093

B. The threshold

Table I reports, for each average degree, the number of configurations in which similarity-based sparsification improves modularity measured on the original graph, and the number in which it improves chance-corrected agreement with the planted communities, out of twelve configurations per degree.

Below average degree 25, the method is harmful on both measures in every configuration tested. At 49.4 it becomes beneficial in two thirds of them. By 101.9 it is beneficial in all twelve configurations on recovery and ten of twelve on the objective. The strongest individual cells occur at aggressive retention: at average degree 49.4 and 15% retention, the partition obtained from the sparsified graph scores +0.005 modularity and +0.152 AMI above the partition obtained from the original graph, and at 220.9 the modularity gain reaches +0.012.

The location of this transition was not chosen by us, and it was not known to us when the experiment was designed. It coincides with the degree at which the original authors reported their method beginning to overtake the unsparsified baseline.

C. Three ways the result could be spurious

a) Run-to-run variation: Metis is not deterministic across option seeds. We repeated the decisive cells with ten partitioner seeds per graph and compared the worst sparsified run against the best unsparsified run. The gain survives that comparison at every degree at or above 49.4, with worst-case modularity differences of +0.001, +0.005 and +0.008, and worst-case AMI differences of +0.134, +0.102 and +0.064. Seed standard deviations are 0.002 in modularity and 0.011 in AMI, so the effect exceeds the noise it must clear by a factor of between two and sixty.

b) The edge budget rather than the selection: Any sparsifier at 15% retention produces a smaller graph that is denser within communities, and a fixed-k partitioner may simply behave better on such a graph. Degree-based and uniform random sparsification, applied to the same

graphs at the same realized retention, do not reproduce the effect at any degree. Their modularity changes run from −0.145 to −0.003, and their recovery changes from −0.480 to +0.032. What produces the gain is the similarity signal used to choose which edges to keep, not the number of edges kept.

c) Granularity: The partitioner takes the community count as an input, and both arms are run with the same value, so the two partitions being compared have identical community counts by construction. This is the artifact that no amount of matching fully excludes on a free-granularity detector, and here it is excluded structurally. We also record cluster-size balance and find it slightly improved rather than degraded under sparsification, so the gain is not purchased by producing more uneven communities.

D. The same graphs, a different algorithm family

Running a free-granularity modularity optimizer on the identical graphs gives a different answer. Modularity gains never exceed the spread of a handful of random restarts. Recovery gains do appear, and they grow with degree, reaching +0.096 AMI, but they do not survive the granularity control. Partitioning the original graph at a resolution tuned to match the sparsified partition's community count recovers the planted communities as well or better in twenty-one of the twenty-nine configurations where the comparison is defined. What appeared to be a benefit of sparsification is the benefit of a finer partition, obtainable without touching the graph.

The variable that decides the outcome is therefore the interaction of density with the detection algorithm's degrees of freedom, not the sparsifier and not the graph alone. A partitioner constrained to return a fixed number of balanced groups is helped, above a density threshold, by having its misleading edges removed. An optimizer free to choose its own granularity already has a cheaper route to the same improvement, and takes it.

E. Why density matters: the denoising account

The original authors attributed their gains to the removal of spurious edges, noting that their strongest results came from an assay network known to contain many false-positive interactions. That explanation makes a testable prediction. If sparsification helps by deleting edges that do not belong to the generative structure, then its benefit should grow with the proportion of such edges.

We inject uniformly random edges into benchmark graphs while holding the planted labels fixed, and measure recovery against those labels. The prediction holds. Recovery gain rises monotonically with the injected fraction in every tested configuration for similarity-based sparsification, with rank correlations of unity between noise level and gain. At average degree 50 and 20% retention, the gain grows from +0.062 with no injected noise to +0.184 when the number of spurious edges equals the number of real ones. Degree-based sparsification shows no such response, which is consistent with its selection signal being a function of degree rather than of local structure.

Two qualifications apply. The gain does not vanish when no noise is injected, so denoising adds to an effect that is already present rather than being its sole cause. And a benchmark graph's noise is random by construction, whereas the spurious edges produced by a real measurement process need not be. The experiment establishes the direction of the mechanism, not its magnitude in the field.

F. What the positive result does not say

Two facts constrain how this finding should be read, and both come from the same experiments.

Sparsification repairs a weaker detector rather than producing the best available partition. At every degree tested, a resolution-tuned modularity optimizer running on the untouched graph recovers the planted communities better than any sparsified pipeline. At average degree 49.4 it reaches 0.956 AMI, against 0.754 for the sparsified fixed-k pipeline. Sparsification lifts the fixed-k partitioner from 0.596 to 0.754, a real and substantial improvement to that algorithm, and still leaves it behind an algorithm that never needed sparsification. The defensible statement is that similarity sparsification repairs a fixed-k cut partitioner on dense graphs, not that it improves community detection.

In our implementation the quality gain is also not a speed gain. Computing exact pairwise similarity costs more than it saves. End to end, the pipeline runs at 1.49× the baseline at average degree 10, 1.01× at 100, and 0.58× at 200, which is slower than not sparsifying at all, even though detection alone accelerates by up to 7.2×. The original speedups were obtained with an approximate similarity computation reported as two orders of magnitude cheaper, measured against clustering runs that took hours. Both cost bases differ from ours by enough to account for the discrepancy without any disagreement about what should be counted.

One part of the original report does not reproduce. At high mixing, where the planted structure is nearly absent and baseline recovery falls to between 0.03 and 0.19, we find no gain under either detector at any degree, whereas the original sweep reports its largest improvement in that regime. Whether this is a genuine non-reproduction or a floor effect of the metrics we use cannot be settled at that mixing level, and we report it unresolved.

VII. Discussion

VIII. Conclusion

References

[1] S. Fortunato, "Community detection in graphs," Physics Reports, vol. 486, no. 3–5, pp. 75–174, 2010.

[2] S. Fortunato and M. E. Newman, "20 years of network community detection," Nature Physics, vol. 18, no. 8, pp. 848–850, 2022.

[3] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," Physical Review E, vol. 69, no. 2, p. 026113, 2004.

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, p. P10008, 2008.
[5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," Scientific Reports, vol. 9, no. 1, pp. 1–12, 2019.
[6] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proceedings of the national academy of sciences, vol. 105, no. 4, pp. 1118–1123, 2008.
[7] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," Internet Mathematics, vol. 6, no. 1, pp. 29–123, 2009.
[8] D. A. Spielman and N. Srivastava, "Graph sparsification by effective resistances," in Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, 2008, pp. 563–568.
[9] A. A. Benczúr and D. R. Karger, "Randomized approximation schemes for cuts and flows in capacitated graphs," SIAM Journal on Computing, vol. 44, no. 2, pp. 290–319, 2015.
[10] V. Satuluri, S. Parthasarathy, and Y. Ruan, "Local graph sparsification for scalable clustering," in Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011, pp. 721–732.
[11] A. Lancichinetti, S. Fortunato, and F. Radicchi, "Benchmark graphs for testing community detection algorithms," Physical Review E, vol. 78, no. 4, p. 046110, 2008.